



UNIVERSIDAD DE
COSTA RICA
Sede del Atlántico

RP Recinto de
Paraíso

UNIVERSIDAD DE COSTA RICA
SEDE DEL ATLÁNTICO - RECINTO PARAÍSO
CARRERA DE INFORMÁTICA EMPRESARIAL
CURSO: IF0009 - Desarrollo de Software IV
PROFESOR: Mag. Jonathan Granados C.
SEMESTRE: II-2026

Cheatsheet: Pruebas Unitarias e Integración en Spring Boot (JUnit 5 Jupiter, Mockito y MockMvc)

Resumen

Esta guía rápida y cheatsheet extensivo resume los conceptos, anotaciones, patrones de prueba y ejemplos prácticos para el desarrollo de pruebas unitarias e integración en el back-end con **Java 21**, **Spring Boot 3.x**, **JUnit 5 Jupiter**, **Mockito 5** y **MockMvc**.

1. Configuración de Maven (`pom.xml`)

```
<dependencies>  
  <!-- Starter para Pruebas Unitarias y de Integración -->
```

```

    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-test</artifactId>
      <scope>test</scope>
    </dependency>
  </dependencies>

  <build>
    <plugins>
      <!-- Ejecución automatizada de pruebas -->
      <plugin>
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-surefire-plugin</artifactId>
        <version>3.2.5</version>
      </plugin>

      <!-- Reporte de Cobertura JaCoCo -->
      <plugin>
        <groupId>org.jacoco</groupId>
        <artifactId>jacoco-maven-plugin</artifactId>
        <version>0.8.11</version>
        <executions>
          <execution>
            <goals><goal>prepare-agent</goal></goals>
          </execution>
          <execution>
            <id>report</id>
            <phase>test</phase>
            <goals><goal>report</goal></goals>
          </execution>
        </executions>
      </plugin>
    </plugins>
  </build>

```

2. Anotaciones Principales de JUnit 5 Jupiter

Anotación	Descripción / Uso
-----------	-------------------

<code>@Test</code>	Denota que un método es un caso de prueba.
<code>@BeforeEach</code>	Se ejecuta antes de CADA método <code>@Test</code> .
<code>@AfterEach</code>	Se ejecuta después de CADA método <code>@Test</code> .
<code>@BeforeAll</code>	Se ejecuta UNA vez antes de todas las pruebas (método <code>static</code>).
<code>@AfterAll</code>	Se ejecuta UNA vez después de todas las pruebas (método <code>static</code>).
<code>@DisplayName("...")</code>	Define un nombre legible para el caso de prueba.
<code>@Disabled("...")</code>	Deshabilita temporalmente la ejecución del test.

3. Principales Aserciones en JUnit 5 (`Assertions.*`)

```
import static org.junit.jupiter.api.Assertions.*;

// Igualdad y valores primarios
assertEquals(esperado, actual);
assertNotEquals(noEsperado, actual);
assertTrue(condicion);
assertFalse(condicion);

// Nulos
assertNull(objeto);
assertNotNull(objeto);

// Captura y verificación de Excepciones
```

```

ResourceNotFoundException ex = assertThrows(
    ResourceNotFoundException.class,
    () -> servicio.buscarPorId(99L)
);
assertEquals("Recurso no encontrado", ex.getMessage());

// Aserciones Agrupadas (assertAll)
assertAll("Validacion de Usuario",
    () -> assertEquals("Carlos", usuario.getNombre()),
    () -> assertEquals("carlos@ucr.ac.cr", usuario.getEmail()),
    () -> assertTrue(usuario.isActivo())
);

```

4. Pruebas Parametrizadas (@ParameterizedTest)

Permiten ejecutar una misma prueba múltiples veces con diferentes argumentos de entrada:

```

import org.junit.jupiter.params.ParameterizedTest;
import org.junit.jupiter.params.provider.ValueSource;
import org.junit.jupiter.params.provider.CsvSource;

// 1. Con lista de valores primarios
@ParameterizedTest
@ValueSource(strings = {"", " ", "\t"})
void validarNombre_CadenasVacias_LanzaExcepcion(String textoInvalido) {
    assertThrows(IllegalArgumentException.class, () -> {
        servicio.validarTexto(textoInvalido);
    });
}

// 2. Con datos tabulares en formato CSV
@ParameterizedTest
@CsvSource({
    "10.0, 0.13, 11.3",
    "100.0, 0.13, 113.0",
    "50.0, 0.0, 50.0"
})
void calcularTotal_ValoresCsv_CalculaCorrectamente(
    double subtotal, double impuesto, double totalEsperado) {
    double resultado = servicio.calcularTotal(subtotal, impuesto);
}

```

```
    assertEquals(totalEsperado, resultado, 0.001);  
}
```

5. Mocks y Stubs con Mockito 5 (org.mockito.*)

```
import org.junit.jupiter.api.extension.ExtendWith;  
import org.mockito.InjectMocks;  
import org.mockito.Mock;  
import org.mockito.junit.jupiter.MockitoExtension;  
import static org.mockito.Mockito.*;  
  
@ExtendWith(MockitoExtension.class)  
class ProductoServiceTest {  
  
    @Mock  
    private ProductoRepository productoRepository;  
  
    @InjectMocks  
    private ProductoService productoService;  
  
    @Test  
    void guardarProducto_Exitoso() {  
        // Arrange (Configurar comportamiento simulado)  
        Producto prod = new Producto(1L, "Laptop", 1200.0);  
        when(productoRepository.save(any(Producto.class)))  
            .thenReturn(prod);  
  
        // Act  
        Producto resultado = productoService.guardar(prod);  
  
        // Assert & Verificacion de Interacciones  
        assertNotNull(resultado);  
        assertEquals("Laptop", resultado.getNombre());  
        verify(productoRepository, times(1)).save(prod);  
        verify(productoRepository, never()).delete(any());  
    }  
}
```

6. Pruebas de la Capa Web con `MockMvc` (`@WebMvcTest`)

```
import org.springframework.boot.test.autoconfigure.web.servlet.WebMvcTest;
import org.springframework.boot.test.mock.mockito.MockBean;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.test.web.servlet.MockMvc;
import org.springframework.http.MediaType;

import static org.mockito.Mockito.when;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.*;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.*;

@WebMvcTest(ProductoController.class)
@AutoConfigureMockMvc(addFilters = false)
class ProductoControllerTest {

    @Autowired
    private MockMvc mockMvc;

    @MockBean
    private ProductoService productoService;

    @Test
    void obtenerPorId_Existe_Returna200yJson() throws Exception {
        Producto prod = new Producto(1L, "Teclado", 45.0);
        when(productoService.obtenerPorId(1L)).thenReturn(prod);

        mockMvc.perform(get("/api/productos/1")
                        .contentType(MediaType.APPLICATION_JSON))
                .andExpect(status().isOk())
                .andExpect(jsonPath("$.id").value(1))
                .andExpect(jsonPath("$.nombre").value("Teclado"));
    }
}
```

7. Consejos y Errores Frecuentes

■ **Consejo Práctico (Patrón AAA):** Divida siempre cada test en **Arrange** (preparar datos y mocks), **Act** (ejecutar el método), y **Assert** (verificar resultados y verificar interacciones de mocks).

■ ■ **Atención (NullPointerException al usar Mocks):** Asegúrese de incluir `@ExtendWith(MockitoExtension.class)` sobre la clase de prueba. Sin esta anotación, `@Mock` e `@InjectMocks` valdrán null.

■ **Importante (Diferencia entre @Mock y @MockBean):** Use `@Mock` en pruebas unitarias puras con Mockito. Use `@MockBean` únicamente en pruebas de contexto Spring como `@WebMvcTest` o `@SpringBootTest`.